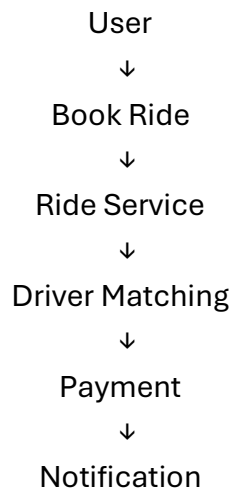


EPIC 09- Task 04: Implement Core System

For Rapido/Bike Taxi core workflow, we are using Java, Spring Boot, Spring Security, REST APIs, API Gateway, Service Discovery, PostgreSQL/MySQL, Redis, Kafka, and Circuit Breaker.

The main workflow is:



1. Create these microservices

Create the following Spring Boot projects:

```
rapido-system/
|
├── api-gateway
├── service-discovery
├── user-service
├── driver-service
├── ride-service
├── payment-service
└── notification-service
```

Infrastructure:

PostgreSQL / MySQL

Redis

Kafka

Step 1 — Service Discovery

Create a **Eureka Server**.

Dependency

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  
</dependency>
```

Main class

```
@SpringBootApplication  
@EnableEurekaServer  
public class ServiceDiscoveryApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(ServiceDiscoveryApplication.class, args);  
    }  
}
```

application.properties

server.port=8761

spring.application.name=service-discovery

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

Run this first.

Open:

<http://localhost:8761>

Step 2 — User Service

Responsibility

- Register users
- Login
- Store user details
- Provide user information

User Entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String phone;
    private String password;

    // getters and setters
}
```

Repository

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserRepository userRepository;

    public UserController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userRepository.save(user);
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));
    }
}
```

Step 3 — Driver Service

Create:
driver-service

Driver entity:

```
@Entity
@Table(name = "drivers")
public class Driver {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;

private String name;
private String phone;
private String vehicleNumber;
private String vehicleType;
private String status;

// getters and setters
}
```

Driver status:

ONLINE
OFFLINE
BUSY

Controller:

```
@RestController
@RequestMapping("/api/drivers")
public class DriverController {

    @GetMapping("/available")
    public String getAvailableDriver() {
        return "Available driver found";
    }
}
```

Later, connect this service to Redis for fast driver availability/location lookup.

Step 4 — Ride Service

This is the **main service** for the business workflow.

Create:

ride-service

Ride entity

```
@Entity
@Table(name = "rides")
public class Ride {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private Long driverId;

    private String pickupLocation;
    private String dropLocation;

    private double fare;

    private String status;

    // getters and setters
}
```

Ride statuses:

```
REQUESTED
DRIVER_ASSIGNED
STARTED
COMPLETED
CANCELLED
```

Book Ride API

```
@PostMapping
public Ride bookRide(@RequestBody Ride ride) {

    ride.setStatus("REQUESTED");
```

```
    return rideRepository.save(ride);  
}
```

API:

POST /api/rides

Example:

```
{  
  "userId": 1,  
  "pickupLocation": "Hitech City",  
  "dropLocation": "Banjara Hills",  
  "fare": 250  
}
```

Step 5 — Driver Matching

After the ride is created:



Example:

```
@Service  
public class DriverMatchingService {  
  
    public Long findDriver() {  
        // Call Driver Service  
        // Find nearest available driver  
        return 101L;  
    }  
}
```

```
}  
}
```

The selected driver ID is stored on the ride.

Step 6 — Payment Service

Create:

payment-service

Payment entity:

```
@Entity  
@Table(name = "payments")  
public class Payment {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private Long rideId;  
    private Long userId;  
    private double amount;  
    private String paymentMethod;  
    private String status;  
  
    // getters and setters  
}
```

Payment API:

POST /api/payments

Example:

```
{  
  "rideId": 1,
```

```
"userId": 1,  
"amount": 250,  
"paymentMethod": "UPI"  
}
```

Response:

```
{  
  "paymentId": 10,  
  "status": "SUCCESS"  
}
```

Step 7 — Kafka

Use Kafka for asynchronous communication.

For example:



Event:

```
{  
  "rideId": 1,  
  "userId": 1,  
  "driverId": 101,  
  "status": "DRIVER_ASSIGNED"  
}
```

Create topic:

ride-events

The Notification Service consumes this event and sends a notification.

Step 8 — Notification Service

Create:

notification-service

Kafka consumer:

```
@KafkaListener(
    topics = "ride-events",
    groupId = "notification-group"
)
public void consumeRideEvent(String message) {

    System.out.println("Ride event received: " + message);

    // Send notification
}
```

Example notification:

Your driver has been assigned.
Driver ID: 101

Step 9 — Redis

Use Redis for fast-access information such as:

Driver location
Driver availability
Temporary ride information
Session/cache data

Example:

driver:101:location

↓

latitude: 17.4483

longitude: 78.3915

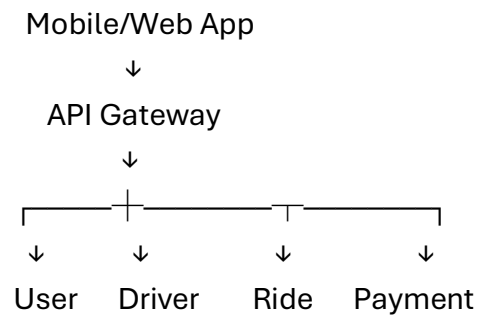
This avoids repeatedly querying the database for rapidly changing location data.

Step 10 — API Gateway

Create:

api-gateway

The client communicates only with the Gateway:



Example routes:

spring:

cloud:

gateway:

routes:

- id: user-service
 - uri: lb://USER-SERVICE
 - predicates:
 - Path=/api/users/**

- id: driver-service
 - uri: lb://DRIVER-SERVICE
 - predicates:

- Path=/api/drivers/**
- id: ride-service
uri: lb://RIDE-SERVICE
predicates:
 - Path=/api/rides/**
- id: payment-service
uri: lb://PAYMENT-SERVICE
predicates:
 - Path=/api/payments/**

Step 11 — Spring Security

Secure the APIs using Spring Security.

Basic rules:

- /api/auth/** → Public
- /api/users/** → Authenticated
- /api/drivers/** → Authenticated
- /api/rides/** → Authenticated
- /api/payments/** → Authenticated

For production implementation, use **JWT authentication**.

Step 12 — Circuit Breaker

Use Resilience4j.

Example:

```
@CircuitBreaker(  
    name = "driverService",  
    fallbackMethod = "driverFallback"  
)
```

```

public String findDriver() {

    return restTemplate.getForObject(
        "http://DRIVER-SERVICE/api/drivers/available",
        String.class
    );
}

```

Fallback:

```

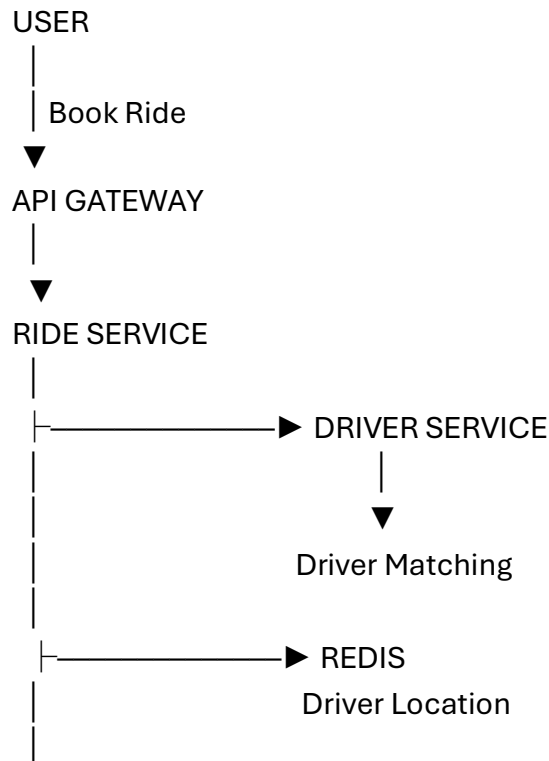
public String driverFallback(Exception exception) {
    return "Driver service is temporarily unavailable";
}

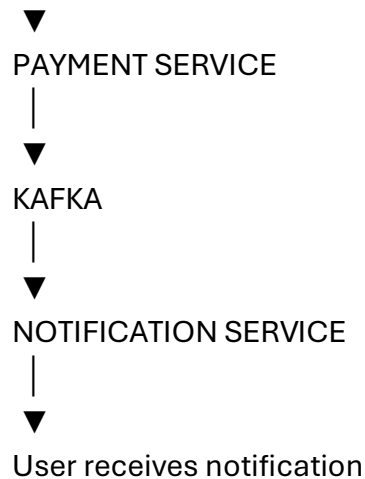
```

This prevents one failed service from bringing down the entire system.

Step 13 — Complete Workflow

Your final implementation should work like this:





Technologies to show in Task 04

Component	Technology
Backend	Java
Framework	Spring Boot
Security	Spring Security + JWT
APIs	REST
Gateway	Spring Cloud Gateway
Discovery	Eureka
Database	PostgreSQL/MySQL
Cache	Redis
Messaging	Kafka
Fault tolerance	Resilience4j Circuit Breaker

For your Eclipse project, start with service-discovery, then create api-gateway, user-service, driver-service, ride-service, payment-service, and notification-service.

This order will make the implementation much easier.